

1. Project overview and submission details

Software Development and Integration

Assignment 3 analytical report. Project: Skin Image Research Tool, a reproducible image-processing module derived from an existing Flask skin-disease demonstration. The work addresses software organization, technology selection, version control, CI/CD, automated testing and scientific software practices.

Submission details

Student: [ADD NAME]. Group: [ADD GROUP]. Date: 9 October 2026. Public repository: [ADD REPOSITORY URL]. Workflow configuration: [ADD WORKFLOW URL]. Successful Actions run: [ADD RUN URL AFTER VALIDATION]. These entries must be completed before submission; no public repository or remote CI execution is claimed by this prepared report.

Problem and scope

The supplied application accepted an image, randomly selected a disease name and probability, and recorded the result. Such outputs do not demonstrate a scientific model. The revised project instead computes deterministic image descriptors and exports them for subsequent research. It does not claim diagnostic capability, model accuracy, early detection or clinical usefulness.

Deliverables and evidence

The package contains a numerical module, a web interface, database storage, CSV export, documentation, an MIT license, issue templates, local Git history and a GitHub Actions configuration. Nine numerical tests passed in the preparation environment. Eleven web integration tests, static lint and the remote workflow remain to be executed after dependency installation. This distinction separates implemented features from demonstrated execution.

Reading guide

Sections 2-4 discuss development organization, technology decisions and version-control alternatives. Sections 5-6 connect CI/CD theory to the prepared workflow. Sections 7-8 describe scientific requirements and the implemented architecture. Sections 9-10 report validation, limitations, conclusions and completion requirements. The report is deliberately scoped to a small coursework research tool, rather than a production medical system.

2. Stages and organization of development

Requirements and acceptance criteria

Development begins by distinguishing the assignment objective from the broader disease-detection ambition. The current objective is a documented scientific module with version control and automation. Acceptance criteria are concrete: valid JPEG/PNG input produces repeatable descriptors; invalid input produces a controlled error; history persists generated records; CSV contains a stable header and numerical values; automated checks block artifact creation when they fail.

Design and task decomposition

A small iterative process is appropriate because the project has one interface, one computation module and a local database. Tasks are separated into numerical processing, request handling, persistence, tests, automation and evidence collection. Each task can be inspected independently. The numerical function should not depend on Flask or SQLite, making it usable later in notebooks or batch scripts.

Implementation and review

Implementation should proceed through short branches with a clear purpose. A feature branch may add CSV export; a separate change may correct image orientation. A reviewer checks behavior, boundary conditions and documentation together. Small commits are easier to understand and reverse than one large final upload. The supplied three commits show preparation stages; subsequent student work should retain its real authorship.

Verification and release

Verification checks both numerical meaning and application behavior. Known synthetic images give expected values, while web tests cover upload errors, lookup behavior and database isolation. A release candidate is prepared only after lint and the full suite pass. The candidate is identified by a Git commit and a workflow run. Deployment, if ever required, would need an additional environment and acceptance procedure.

Maintenance and feedback

Issues record reproducible problems, future features and acceptance criteria. Closing an issue should refer to an actual change or verification result. Maintenance includes dependency updates, rerunning tests, documenting method changes and changing the analysis version when feature semantics change. For this coursework, the immediate feedback cycle ends with a public repository, a successful workflow and a completed report; future clinical development remains a separate project.

3. Criteria and justification for technology selection

Decision criteria

Technology selection considers fit to the task, existing code, ease of installation, reproducibility, maintenance burden, testing support and available time. More libraries are not automatically better. A lightweight stack is appropriate when the deliverable is a local research prototype rather than a distributed service. The chosen tools preserve the useful Flask interface while replacing ungrounded prediction logic.

Language and web framework

Python is selected because the original code is written in it and the scientific computation can remain independently reusable. Flask keeps the existing request/template structure and avoids rewriting the interface in another language. An application factory enables separate test instances with different database paths [2]. Django would be reasonable for a larger application requiring extensive account and administration features; those features would add unnecessary scope here.

Image library and persistence

Pillow supplies decoding, EXIF orientation handling and whole-image statistics [4,5]. OpenCV would be a sensible alternative for segmentation and spatial feature extraction, but those operations are outside this version. SQLite provides local persistence without a managed server. PostgreSQL would become more attractive for concurrent users, access policies and larger deployments. CSV is a simple interoperable format for downstream exploratory analysis.

Testing, lint and delivery

Standard-library unittest is selected to avoid an extra testing dependency. Its assertions are sufficient for numerical fixtures and Flask client checks. Pytest is a useful alternative when fixtures and parameterization become more complex. Ruff is configured for common syntax, import and undefined-name errors. GitHub Actions is selected because the assignment requires GitHub hosting, so checks can live beside the repository rather than on a separately administered server.

Environment tradeoffs

Python 3.11 and 3.12 are the configured CI targets. Direct dependency versions are recorded, but the file is not a complete transitive lock. A virtual environment limits interference from unrelated projects. Library versions should be reviewed before long-term deployment. Here the decisive tradeoff is a small, explainable stack with a numerical module that can later be integrated into a larger research workflow.

4. Version-control systems and GitHub workflow

What version control records

Version control records file revisions and supports inspection, comparison and recovery. Git stores distributed repository history, allowing local commits and branches without a central connection. Subversion uses a centralized model, while Mercurial is another distributed system. These are version-control systems. GitHub is a hosting and collaboration platform for Git repositories, so comparing Git and GitHub as competing systems would confuse two different roles [1].

Comparison for this project

Git is chosen for this assignment because the repository must be hosted on GitHub and its branches, commits and remote exchange support an inspectable workflow. Subversion could suit a team organized around a central repository, but it would not satisfy the required Git workflow. Mercurial could also preserve distributed history, yet using it here would add a hosting mismatch without a scientific benefit. The comparison therefore follows project constraints rather than claiming one tool is best in every setting.

Practical organization

The main branch contains the current submission candidate. Changes should be developed on short branches, committed with descriptive messages and proposed through pull requests. A pull request links the rationale, issue and check results. A failed check should be fixed before integration. Repository settings can enforce review and passing checks where available, but no such setting is claimed to be configured in this prepared package.

Tracked and excluded content

Source, templates, tests, documentation, license and workflow are tracked. Virtual environments, caches, local databases and secrets are ignored. The previous archive contained uploaded images and an existing database; these are excluded from the revised submission. The new module does not save source photographs. This keeps the repository focused on reproducible code and avoids mixing private or unrelated runtime data with software history.

Evidence of correct use

The delivered local repository has meaningful preparation commits rather than a fabricated past development timeline. Actual GitHub hosting is still pending. Submission evidence should include the public repository URL, its commit history, actual issues and a successful Actions run. Issue-template files support consistent reporting, but they do not create GitHub issues by themselves. A student should understand these distinctions when demonstrating the project.

5. Key concepts and practices of CI/CD

Continuous integration

Continuous integration combines frequent changes with automated checks. Its purpose is to reveal regressions soon after a change is proposed. A check must exercise meaningful behavior: parsing valid images, rejecting invalid ones and calculating expected values are stronger evidence than merely importing a file. Static checking complements execution by detecting errors in paths that a test may not cover.

Delivery versus deployment

Continuous delivery prepares a checked version that can be released through a later decision. Continuous deployment additionally performs an automated release to a running environment. GitHub documents deployment workflows and the storage of workflow artifacts as distinct capabilities [6,7]. This project implements integration and source-artifact delivery. It does not automatically start a hosted application, publish a clinical service or prove production readiness.

A quality gate

A quality gate is a condition that must succeed before the next stage proceeds. In this project, dependency installation, lint, tests and compilation precede source packaging. If an earlier step fails, the normal job sequence stops and the artifact is not created. The workflow therefore connects delivered source to a successful check sequence rather than producing a ZIP regardless of errors.

Practical principles

Automation should be understandable, sufficiently fast and reproducible enough to rerun. Tests should not rely on private files or network datasets. Logs should explain failures without containing confidential data. Build artifacts should identify the checked source revision. A version matrix helps detect interpreter differences but costs additional runner time. Two supported interpreter versions are a reasonable small-project compromise.

Limits of automated assurance

A green workflow confirms that configured checks passed in the recorded environment. It does not establish medical validity, complete security or the absence of every defect. Source packaging is also not a full deployment: dependencies remain necessary at runtime. Scientific conclusions require separate study design and evidence. The report therefore treats CI as software-process evidence, not as experimental evidence for disease detection.

6. Implementation of the GitHub Actions pipeline

Triggers and configuration

The configuration is stored at `.github/workflows/ci.yml`. It responds to pushes, pull requests and manual workflow dispatches. Each job receives repository read permission. The matrix requests Python 3.11 and 3.12 on an Ubuntu runner. GitHub recommends `setup-python` to select interpreter versions consistently across runners [3]. The pipeline configuration is ready for publication but has not yet been executed remotely.

Validation steps

The runner checks out the repository, selects Python and installs `requirements-dev.txt`. Ruff checks the source. `unittest discovery` then runs nine numerical tests and eleven application tests. `Python compileall` verifies source syntax. These steps target different failure types: static errors, numerical regressions, web behavior and syntax validity. No success claim is made merely because the YAML contains these commands.

Artifact preparation

After validation, `git archive` writes a ZIP from the checked commit rather than copying the working directory. This excludes the local database, virtual environment and unrelated untracked files. `upload-artifact` delivers the archive under a name containing the interpreter version [6]. The two matrix jobs therefore provide separately checked source packages. The archive is source delivery and still requires runtime dependency installation.

Observing a real run

After the repository is pushed, the student must open Actions and inspect both jobs. Any failure should be addressed and rerun. A valid demonstration records the commit ID, final successful run URL and downloadable artifact. If the workflow does not start, verify that the YAML is in the repository root path and Actions is enabled. If dependency installation fails, inspect the runner log before changing application code.

Submission links and future improvements

The completed report must link to the public repository, workflow file and successful run. Suggested future improvements include immutable action references, a fully resolved dependency lock and structured test reports. Authentication, secrets, a deployment target and rollback behavior would be necessary for an actual hosted release. They are deliberately not claimed as implemented features of the coursework tool.

7. Requirements and best practices for scientific software

Repeatability and provenance

A scientific tool should make its numerical operations explicit. The feature extractor receives a binary stream, decodes the image, applies its orientation, converts to RGB and calculates statistics. The output records `analysis_version=whole-image-v1`. SQLite adds a generated record identifier and a UTC timestamp. These fields document the method version and execution record, but do not identify the original dataset or image content.

Environment and data management

A virtual environment and dependency file describe the runtime setup. CI checks two interpreter versions. For a publication-quality experiment, a fuller environment lock and dataset provenance would be required. This prototype does not preserve input files or hashes; identical records cannot by themselves prove identical input data. Researchers must maintain an approved external dataset manifest if later experiments need input traceability.

Numerical meaning

The mean of a channel is the sum of its pixel values divided by the pixel count. Brightness is the mean of the converted grayscale image. Contrast is its population standard deviation, the square root of the average squared deviation from the mean. Pillow provides the mean and standard-deviation statistics used here [4]. Values are rounded to four decimal places for a readable export, while computation occurs before rounding.

Testing scientific assumptions

Synthetic images isolate assumptions more reliably than a collection of unexplained photographs. A constant-color image tests channel means. A black/white two-pixel image tests brightness and population contrast. EXIF rotation tests the dimensional convention. Repeatability tests exclude the original random behavior. These checks validate the stated computation, not the suitability of these features for disease classification.

Research limitations and responsible extension

Whole-image means include background and depend on lighting and acquisition. They cannot distinguish disease classes on their own. A future classifier would require an appropriately licensed dataset, leakage controls, a justified baseline and measured held-out performance. Subject-level splitting, model provenance and bias assessment would be part of that separate study. The current contribution is software organization and a reproducible preprocessing building block.

8. Architecture, input validation and documentation

Separation of responsibilities

The project separates computation from infrastructure. `analysis.py` contains `extract_features`. `app.py` creates the Flask application and handles upload, result lookup, history and export. Templates display the data, and SQLite persists a compact record. This separation allows the numerical module to be called directly from another Python program and tested without launching a web server.

Data flow and persistence

The browser sends a multipart request to `/upload`. The application processes the stream in memory, generates a UUID and inserts its timestamp and feature JSON through a parameterized SQL statement. It redirects to `/result/<record_id>`, which reads the stored result. History lists existing records, and `/export.csv` writes a fixed field order. Result values are retrieved from the database rather than accepted from arbitrary URL query parameters.

Input controls

The request body is limited to 8 MB and decoded images to 20 million pixels. Accepted content is JPEG or PNG, determined from decoded format rather than filename extension alone. Pillow errors and decompression-bomb warnings are converted to controlled invalid-input responses. The application never uses the uploaded filename as a filesystem path. This removes the overwrite and path-handling concerns of the original save-by-filename approach.

Local use and limits

The server binds to 127.0.0.1 with debug mode disabled. There is no authentication, CSRF protection or multi-user isolation. The tool is therefore scoped to local coursework with synthetic or non-sensitive images. A publicly reachable deployment would require a different security and operational design. Publishing the source repository does not imply publishing the running application or its database.

Documentation as part of quality

The README explains installation, feature definitions, testing, architecture and limitations. The license describes software reuse terms without granting dataset rights. The submission guide provides publication and evidence steps. Issue templates request environment information and acceptance criteria. Keeping these documents beside the implementation makes a reviewer able to inspect the actual behavior rather than relying on a presentation alone.

9. Validation, self-analysis and challenges

Executed checks

Preparation used Python 3.12 and an already installed Pillow library. Nine numerical tests passed: known RGB means, constant grayscale endpoints, nonuniform population contrast, repeatability, JPEG support, invalid bytes, GIF rejection, pixel-limit rejection and EXIF orientation. Source compilation also passed. The preparation library environment was not the complete pinned requirements environment, so these results are deliberately reported narrowly.

Checks still pending

Flask and Ruff were not installed in the preparation environment and package downloads were unavailable. Consequently the eleven web integration tests and Ruff were not executed there. GitHub Actions has not been run because a public repository has not yet been published. The student must install the requirements, run all twenty tests and lint, then confirm the Python 3.11 and 3.12 workflow jobs before presenting a final success statement.

Web-test intent

The prepared integration tests check the home page and empty history, upload/result/history/CSV flow, missing uploads, empty filenames, fake image bytes, request size rejection, unknown record lookup, repeated filenames, filename non-persistence, database isolation and the empty CSV header. These are meaningful behavior checks, but prepared test code is not the same as an observed passing run.

Challenges and resolutions

The original random diagnosis was the main conceptual weakness. It was removed instead of presenting it as model inference. Database creation originally depended on direct execution; it is now part of the application factory. Arbitrary saved filenames were replaced by in-memory processing and generated record IDs. Limited access to dependencies prevented complete runtime validation and is recorded openly rather than hidden in the report.

Self-analysis

The revised scope is achievable and defensible for Assignment 3: it demonstrates a scientific computation module and an organized software workflow. Its strongest evidence is explicit numerical semantics and verified synthetic cases. Its weakest current evidence is the missing full-environment and remote validation. The next action is evidence completion, not adding unvalidated complexity. No exact grade is predicted because final publication and demonstration remain the student responsibility.

10. Conclusions, completion checklist and references

Conclusion

The project was converted from a random-output demonstration into a deterministic image-processing prototype. It now has documented numerical features, SQLite history, CSV export, tests, local Git commits and an automation configuration that delivers source only after checks. This satisfies the implementable part of the assignment scope while keeping scientific and execution claims separate.

Completion before submission

Add student and group details. Publish the repository publicly. Run the full tests and lint. Confirm both Actions jobs and inspect the source artifacts. Open actual GitHub issues and close the verification issue with evidence. Insert the repository, workflow and successful run links. Update Section 9 with observed results. Export the final report in Times New Roman, 12 pt, 1.5 spacing and justified alignment; verify 8-12 pages.

Formatting note

This supplied PDF uses the available standard Times-Roman face. The editable HTML requests Times New Roman explicitly. For literal compliance, export that HTML to PDF on a computer with Times New Roman installed. Recheck pagination after replacing submission links; no successful publication or complete CI run should be invented.

References (official documentation)

[1] Git, Pro Git: About Version Control.

<https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control.html>

[2] Flask: Application Factories. <https://flask.palletsprojects.com/en/stable/patterns/appfactories/>

[3] GitHub Docs: Building and testing Python.

<https://docs.github.com/en/actions/tutorials/build-and-test-code/python>

[4] Pillow: ImageStat module. <https://pillow.readthedocs.io/en/stable/reference/ImageStat.html>

[5] Pillow: ImageOps module. <https://pillow.readthedocs.io/en/stable/reference/ImageOps.html>

[6] GitHub Docs: Workflow artifacts.

<https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>

[7] GitHub Docs: Continuous deployment.

<https://docs.github.com/en/actions/get-started/continuous-deployment>

Documentation consulted on 9 October 2026. Sources support tool concepts; implementation descriptions and limitations are based on the delivered code and preparation checks.